

本周五（27日）有上机课

- 18:00-20:15
- 理工配楼二层 1、2 机房

第 3 讲：表格数据处理与 Pandas - 2

“数据科学导论”专业核心课



范举教授

fanj@ruc.edu.cn

上节课程回顾：Pandas 的数据结构

- 在 Pandas 中，我们称一个表格为一个 DataFrame
- 一个 DataFrame 由一组列数据组成，每列成为一个 Series

	Name	Sex	Count	Year	Century
0	mary	F	7065	1880	19th Century
1	anna	F	2604	1880	19th Century
2	emma	F	2003	1880	19th Century
3	elizabeth	F	1939	1880	19th Century
4	minnie	F	1746	1880	19th Century
...	...	...	...	...	...
1957041	zylas	M	5	2018	21st Century
1957042	zyran	M	5	2018	21st Century
1957043	zyrie	M	5	2018	21st Century
1957044	zyron	M	5	2018	21st Century
1957045	zzyzx	M	5	2018	21st Century

A
DataFrame

```
0      7065
1      2604
2      2003
3      1939
4      1746
...
1957041      5
1957042      5
1957043      5
1957044      5
1957045      5
Name: Count, Length: 1957046, dtype: int64
```

A Series named "Count"

上节课程回顾：Series

- Series 类似一维数组，由一组**数据**和对应的**索引**组成
 - 数据值（**values**）：同一数据类型的一组数值
 - 索引（**index**）：标记数据值位置（或其它）的一组标签

```
import pandas as pd
s = pd.Series(["welcome", "to", "data 100"])
```

0	welcome
1	to
2	data 100

dtype: object

Index, accessed by calling **s.index**

➤ RangeIndex(start=0, stop=3, step=1)

• **Values**, accessed by calling

s.values

➤ array(['welcome', 'to', 'data 100'], dtype=object)



思考：Series 和 Python 的基本数据类型 List 有何异同

上节课程回顾：Series 对象的创建

- 请思考如何创建以下的 Series 对象？



obj1

```
0    4
1    7
2   -5
3    3
dtype: int64
```

obj2

```
d    4
b    7
a   -5
c    3
dtype: int64
```

obj3

```
Ohio    35000
Texas   71000
Oregon  16000
Utah     5000
dtype: int64
```

```
obj1 = pd.Series([4, 7, -5, 3])
```

```
obj2 = pd.Series(data = [4, 7, -5, 3], index=['d', 'b', 'a', 'c'])
```

```
sdata = {'Ohio': 35000, 'Texas': 71000, 'Oregon': 16000, 'Utah': 5000}
obj3 = pd.Series(sdata)
```

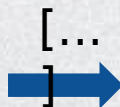
上节课程回顾：DataFrame

- DataFrame
 - 二维的表格型数据结构
 - 可以将 DataFrame 理解为一组共享 **Index** 信息的 Series 对象

```
0    1824
1    1824
2    1828
3    1828
4    1832
...
177   2016
178   2020
179   2020
180   2020
181   2020
Name: Year,
```



```
0    Andrew Jackson
1    John Quincy Adams
2    Andrew Jackson
3    John Quincy Adams
4    Andrew Jackson
...
177   Jill Stein
178   Joseph Biden
179   Donald Trump
180   Jo Jorgensen
181   Howard Hawkins
Name: Candidate,
```



	Year	Candidate	Party	Popular vote	Result	%
0	1824	Andrew Jackson	Democratic-Republican	151271	loss	57.210122
1	1824	John Quincy Adams	Democratic-Republican	113142	win	42.789878
2	1828	Andrew Jackson	Democratic	642806	win	56.203927
3	1828	John Quincy Adams	National Republican	500897	loss	43.796073
4	1832	Andrew Jackson	Democratic	702735	win	54.574789
...	...	...	...	...	...	...
177	2016	Jill Stein	Green	1457226	loss	1.073699
178	2020	Joseph Biden	Democratic	81268924	win	51.311515
179	2020	Donald Trump	Republican	74216154	loss	46.858542
180	2020	Jo Jorgensen	Libertarian	1865724	loss	1.177979
181	2020	Howard Hawkins	Green	405035	loss	0.255731

The Series
"Year"

The Series "Candidate"

The DataFrame
elections

DataFrame 的数据筛选操作

考虑我们创建了
下述
DataFrame 对象
weird

```
weird = pd.DataFrame({
    1: ["topdog", "botdog"],
    "1": ["topcat", "botcat"]
})
weird
```

	1	1
0	topdog	topcat
1	botdog	botcat

- 请给出以下代码的输出结果：

	1	1
0	topdog	topcat
1	botdog	botcat

	1
0	topdog
1	botdog

Name: 1, dtype: object

- weird[1]

	1	1
0	topdog	topcat
1	botdog	botcat

	1
0	topcat
1	botcat

Name: 1, dtype: object

- weird["1"]

	1	1
0	topdog	topcat
1	botdog	botcat

	1
1	botdog

- weird[1:]

	1	1
0	topdog	topcat
1	botdog	botcat

	1	1
0	topcat	topdog
1	botcat	botdog

- weird[["1", 1]]

```
-> 3654 raise KeyError(key) from err
3655 except TypeError:
3656     # If we have a listlike key, _check_indexing_er
3657     # InvalidIndexError. Otherwise we fall through
3658     # the TypeError.
3659     self._check_indexing_error(key)
```

KeyError: 0

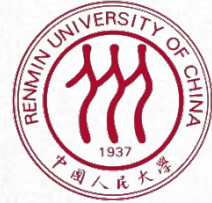
- weird.loc[:, 0]

	1	1
0	topdog	topcat
1	botdog	botcat

	1
1	botdog
1	botcat

Name: 1, dtype: object

- weird.loc[1]



提纲



表格数据处理与 Pandas

1. 条件数据筛选
2. 常用数据处理操作
3. 数据分组与数据透视表
4. 案例分析与课堂练习

我们已经学过的数据筛选操作

- 基于标签的数据筛选 (`.loc`) : 特定 column 或 index **标签**的数据
- 基于整数的数据筛选 (`.iloc`) : 特定 column 或 index **位置**的数据

```
[3]: babynames = pd.read_csv('./babynames.csv')
babynames
```

```
[3]:
```

	Name	Sex	Count	Year
0	Mary	F	7065	1880
1	Anna	F	2604	1880
2	Emma	F	2003	1880
3	Elizabeth	F	1939	1880
4	Minnie	F	1746	1880
...	...	...	...	...
1957041	Zylas	M	5	2018
1957042	Zyran	M	5	2018
1957043	Zyrie	M	5	2018
1957044	Zyron	M	5	2018
1957045	Zzyzx	M	5	2018

1957046 rows x 4 columns

```
babynames_first_10_rows = babynames.loc[:9,
```

```
]
```

	Name	Sex	Count	Year
0	Mary	F	7065	1880
1	Anna	F	2604	1880
2	Emma	F	2003	1880
3	Elizabeth	F	1939	1880
4	Minnie	F	1746	1880
5	Margaret	F	1578	1880
6	Ida	F	1472	1880
7	Alice	F	1414	1880
8	Bertha	F	1320	1880
9	Sarah	F	1288	1880

如何筛选
出奇数行

	Name	Sex	Count	Year
0	Mary	F	7065	1880
1	Anna	F	2604	1880
2	Emma	F	2003	1880
3	Elizabeth	F	1939	1880
4	Minnie	F	1746	1880
5	Margaret	F	1578	1880
6	Ida	F	1472	1880
7	Alice	F	1414	1880
8	Bertha	F	1320	1880
9	Sarah	F	1288	1880

使用布尔数组进行数据筛选

- 筛选方法 `loc` 和 `[]` 可以接收一个布尔数组作为输入
- 筛选规则：索引对应为 `True` 的行返回，对应为 `False` 的不返回

	Name	Sex	Count	Year
0	Mary	F	7065	1880
1	Anna	F	2604	1880
2	Emma	F	2003	1880
3	Elizabeth	F	1939	1880
4	Minnie	F	1746	1880
5	Margaret	F	1578	1880
6	Ida	F	1472	1880
7	Alice	F	1414	1880
8	Bertha	F	1320	1880
9	Sarah	F	1288	1880

```
babynames_first_10_rows[[True, False, True, False,  
True, False, True, False, True, False]]
```

```
babynames_first_10_rows.loc[[True, False, True,  
False, True, False, True, False, True, False], :]
```

	Name	Sex	Count	Year
0	Mary	F	7065	1880
2	Emma	F	2003	1880
4	Minnie	F	1746	1880
6	Ida	F	1472	1880
8	Bertha	F	1320	1880



思考：筛选方法 `iloc` 是否也可如此？

使用布尔数组进行数据筛选

- 使用**逻辑表达式**产生布尔数组，进而进行条件数据筛选

```
cond = (babynames["Sex"] == "F") & (babynames["Year"] >
```

```
2000)
```

```
0      False
1      False
2      False
3      False
4      False
```

```
...
```

```
1957041  False
1957042  False
1957043  False
1957044  False
1957045  False
```

```
Length: 1957046, dtype: bool
```

一个由布尔
值构成的
Series 对象

`babynames[cond]`

`babynames.loc[cond, :]`

`babynames.iloc[cond, :]`

	Name	Sex	Count	Year
1362582	Emily	F	25057	2001
1362583	Madison	F	22164	2001
1362584	Hannah	F	20716	2001
1362585	Ashley	F	16527	2001
1362586	Alexis	F	16402	2001
...	...	...	...	...
1943037	Zymirah	F	5	2018
1943038	Zynah	F	5	2018
1943039	Zyniyah	F	5	2018
1943040	Zynlee	F	5	2018
1943041	Zyona	F	5	2018

345415 rows x 4 columns

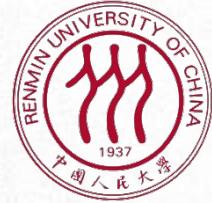
使用布尔数组进行数据筛选

- 使用**逻辑表达式**产生布尔数组，进而进行条件数据筛选
- 如何连接多个逻辑表达式？
 - 考虑 p 和 q 是两个布尔数组或是取值为布尔值的 Series 对象

Symbol	Usage	Meaning
~	~p	Negation of p
	p q	p OR q
&	p & q	p AND q
^	p ^ q	p XOR q (exclusive or)



课堂练习



- 返回 Name 中包含 Angela、Jacob、Zekai、Maya 的记录

	Name	Sex	Count	Year
0	Mary	F	7065	1880
1	Anna	F	2604	1880
2	Emma	F	2003	1880
3	Elizabeth	F	1939	1880
4	Minnie	F	1746	1880
...	...	...	...	...
1957041	Zylas	M	5	2018
1957042	Zyran	M	5	2018
1957043	Zyrie	M	5	2018
1957044	Zyron	M	5	2018
1957045	Zzyzx	M	5	2018

1957046 rows x 4 columns



```
babynames[(babynames["Name"] == "Angela") |  
(babynames["Name"] ==  
"Jacob") |  
(babynames["Name"] ==  
"Zekai") |  
(babynames["Name"] ==  
"Maya")]  
  
babynames[(babynames["Name"]  
).isin(["Angela", "Jacob", "  
Zekai", "Maya"])]
```

	Name	Sex	Count	Year
682	Angela	F	7	1880
980	Jacob	M	404	1880
2583	Angela	F	9	1881
2976	Jacob	M	381	1881
4295	Angela	F	24	1882
...	...	...	...	...
1925073	Maya	F	3882	2018
1925276	Angela	F	1236	2018
1935848	Jacob	F	9	2018
1943054	Jacob	M	11770	2018
1951807	Zekai	M	9	2018

518 rows x 4 columns



课堂练习



- 返回所有 Name 以 M 开头的数据记录

```
babynames[babynames["Name"].str.startswith("M")]
```

```
0      True
1     False
2     False
3     False
4      True
...
1957041 False
1957042 False
1957043 False
1957044 False
1957045 False
Name: Name, Length: 1957046, dtype: bool
```

	Name	Sex	Count	Year
0	Mary	F	7065	1880
4	Minnie	F	1746	1880
5	Margaret	F	1578	1880
15	Martha	F	1040	1880
20	Maude	F	859	1880
...	...	...	...	...
1956456	Musiq	M	5	2018
1956457	Muzzammil	M	5	2018
1956458	Mykhail	M	5	2018
1956459	Mykol	M	5	2018
1956460	Myzell	M	5	2018

思考：还有哪些字符串函数可以用于数据筛选



课堂练习

babynames

	Name	Sex	Count	Year
0	Mary	F	7065	1880
1	Anna	F	2604	1880
2	Emma	F	2003	1880
3	Elizabeth	F	1939	1880
4	Minnie	F	1746	1880
...	...	...	...	...
1957041	Zylas	M	5	2018
1957042	Zyran	M	5	2018
1957043	Zyrie	M	5	2018
1957044	Zyron	M	5	2018
1957045	Zzyzx	M	5	2018

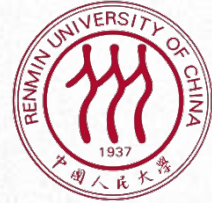
1957046 rows x 4 columns

- 返回在 2000 年，起名为 Alex 的女性新生儿有多少个（注：只返回个数即可）

```
babynames[(babynames["Name"] == "Alex") &
            (babynames["Sex"] == "F") &
            (babynames["Year"] ==
2000)].iloc[0, 3]
```

- 返回姓名以 Kev 开头或者 Count 大于 600 的所有新生儿姓名记录

```
babynames[(babynames["Name"].str.startswith("Kev
")) |
            (babynames["Count"] > 600)]
```



提纲



表格数据处理与 Pandas

1. 条件数据筛选
2. 常用数据处理操作
3. 数据分组与数据透视表
4. 案例分析与课堂练习



如何添加一个新的 Column ?

- 请给 babynames 对象添加一个新列：name_length

babynames

	Name	Sex	Count	Year
0	Mary	F	7065	1880
1	Anna	F	2604	1880
2	Emma	F	2003	1880
3	Elizabeth	F	1939	1880
4	Minnie	F	1746	1880
...	...	...	...	...
1957041	Zylas	M	5	2018
1957042	Zyran	M	5	2018
1957043	Zyrie	M	5	2018
1957044	Zyron	M	5	2018
1957045	Zzyzx	M	5	2018

1957046 rows x 4 columns

直接使用 **[]** 来引用
新列，并为其初始化



```
babynames["name_length"] =  
babynames["Name"].str.len()
```

	Name	Sex	Count	Year	name_length
0	Mary	F	7065	1880	4
1	Anna	F	2604	1880	4
2	Emma	F	2003	1880	4
3	Elizabeth	F	1939	1880	9
4	Minnie	F	1746	1880	6
...	...	...	...	...	...
1957041	Zylas	M	5	2018	5
1957042	Zyran	M	5	2018	5
1957043	Zyrie	M	5	2018	5
1957044	Zyron	M	5	2018	5
1957045	Zzyzx	M	5	2018	5

1957046 rows x 5 columns

Pandas 的常用数据操作

- 整体修改某一个 column 中的值

```
# Modify the "name_lengths" column to be one less than its original value
```

```
babynames["name_lengths"] = babynames["name_lengths"] - 1
```

- 为某列重命名

```
# Rename "name_lengths" to "Length"
```

```
babynames = babynames.rename(columns={"name_lengths": "Length"})
```

- 删除某列 (drop 函数默认是删除行)

```
babynames = babynames.drop("Length", axis="columns")
```

axis= "columns"



课堂练习

- 请选择下列语句的返回结果

```
babynames.drop("name_length",  
axis="columns")
```

babynames

	Name	Sex	Count	Year	name_length
0	Mary	F	7065	1880	4
1	Anna	F	2604	1880	4
2	Emma	F	2003	1880	4
3	Elizabeth	F	1939	1880	9
4	Minnie	F	1746	1880	6
...	...	...	...	...	...
1957041	Zylas	M	5	2018	5
1957042	Zyran	M	5	2018	5
1957043	Zyrie	M	5	2018	5
1957044	Zyron	M	5	2018	5
1957045	Zzyzx	M	5	2018	5

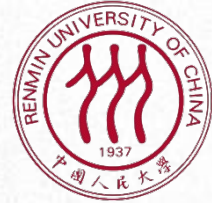
1957046 rows x 5 columns



	Name	Sex	Count	Year
0	Mary	F	7065	1880
1	Anna	F	2604	1880
2	Emma	F	2003	1880



	Name	Sex	Count	Year	name_length
0	Mary	F	7065	1880	4
1	Anna	F	2604	1880	4
2	Emma	F	2003	1880	4



使用 NumPy 函数

- Pandas 的 Series 与 DataFrame 对象支持 NumPy 函数
- 请给出名为 John 的新生儿数量的最小、最大、平均值、标准差

生成满足条件的 Series 对象

```
john_count = babynames[babynames["Name"] == "John"]["Count"]
```

```
228      46
942    9655
2312     26
2938   8769
4212     40
...
1878276 10013
1902691    10
1910824   9486
1933299    13
1943068   9119
Name: Count, Length: 278, dtype: int64
```

```
john_count.max()
88319

john_count.min()
10

john_count.mean()
18512.61870503597

john_count.std()
25711.2928004684
```

扩展阅读：**NumPy** 教程

<https://www.runoob.com/numpy/numpy-tutorial.html>

使用 Pandas 的内置函数

- Pandas 提供了一系列非常有用的内置函数
 - size/shape # 返回 DataFrame 对象的元素数 / 行列数
 - describe # 返回 DataFrame 对象的统计信息

```
babynames.shape
```

```
(1957046, 4)
```

```
babynames.size
```

```
7828184
```

```
babynames.describe()
```

	Count	Year
count	1957046.00	1957046.00
mean	179.69	1975.56
std	1522.80	34.19
min	5.00	1880.00
25%	7.00	1952.00
50%	12.00	1985.00
75%	32.00	2004.00
max	99689.00	2018.00

```
babynames.describe(include=['O'])
```

	Name	Sex
count	1957046	1957046
unique	98400	2
top	Lee	F
freq	278	1156527

使用 Pandas 的内置函数

- Pandas 提供了一系列非常有用的内置函数
 - `value_counts` # 返回每个取值出现的频次，Series 对象
 - `unique` # 返回不同的取值，数组对象

```
babynames["Name"].value_counts(  
)
```

```
babynames["Name"].unique()
```

```
babynames["Name"].unique()  
array(['Mary', 'Anna', 'Emma', ..., 'Zihir', 'Zin', 'Zuber'], dtype=object)
```

```
babynames["Name"].value_counts()
```

```
James      278  
Lee        278  
Johnnie    278  
William    278  
Joseph     278  
...  
Sugeily    1  
Taniayah   1  
Jainiyah   1  
Chejuana   1  
Babbett    1  
Name: Name, Length: 98400, dtype: int64
```

使用 Pandas 的内置函数

- 人为给定顺序
 - `reindex`
- 按照索引自动排序
 - `sort_index()`
- 按照变量自动排序
 - `sort_values( by = " )`

```
df1 = nba_df.sort_index (axis=0, ascending=False)
df1.head()
```

	Team	Win	Loss
7	Utah Jazz	13	14
6	New Orleans Pelicans	14	13
5	Portland Trail Blazers	13	12
4	Denver Nuggets	14	12
3	Minnesota Timberwolves	16	11

```
df2 = nba_df.sort_index (axis=1, ascending=False)
df2.head()
```

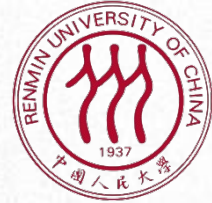
	Win	Team	Loss
0	20	Houston Rockets	4
1	21	Golden State Warriors	6
2	19	San Antonio Spurs	8
3	16	Minnesota Timberwolves	11
4	14	Denver Nuggets	12

```
nba_df = nba_df.sort_values(by='Win')
nba_df.head()
```

	Team	Win	Loss
5	Portland Trail Blazers	13	12
7	Utah Jazz	13	14
4	Denver Nuggets	14	12
6	New Orleans Pelicans	14	13
3	Minnesota Timberwolves	16	11



课堂练习



- 将 babynames 对象按照 Name 列的长度从大到小排序
 - 方法一：创建一个临时列 name_length 再进行排序

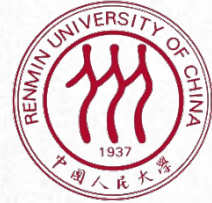
```
babynames['name_length'] = babynames['Name'].str.len()  
babynames.sort_values(by='name_length', ascending=False)
```

	Name	Sex	Count	Year	name_length
1422766	Matthewalexande	M	5	2002	15
882411	Christopherjohn	M	5	1980	15
1550162	Christopherjohn	M	6	2006	15
1170138	Joshuaalexander	M	5	1993	15
1067541	Christopherjohn	M	6	1989	15
...	...	...	...	...	...
1065210	Sy	M	11	1989	2
110886	Le	M	7	1913	2
1065123	La	M	11	1989	2
1065095	Jt	M	11	1989	2
655349	Wm	M	13	1966	2

1957046 rows x 5 columns



课堂练习



- 将 babynames 对象按照 Name 列的长度从大到小排序
 - 方法二：使用 sort_values 的 Key 参数

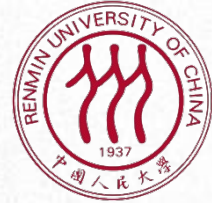
```
babynames.sort_values("Name", key=lambda x: x.str.len(),  
                      ascending=False)
```

	Name	Sex	Count	Year
1422766	Matthewalexande	M	5	2002
882411	Christopherjohn	M	5	1980
1550162	Christopherjohn	M	6	2006
1170138	Joshuaalexander	M	5	1993
1067541	Christopherjohn	M	6	1989
...	...	...	...	...
1065210	Sy	M	11	1989
110886	Le	M	7	1913
1065123	La	M	11	1989
1065095	Jt	M	11	1989
655349	Wm	M	13	1966

1957046 rows x 4 columns



课堂练习



- 将 babynames 对象按照包含 "ea" 的次数排序

```
def ea_count(string):  
    return string.count('ea')  
  
babynames.sort_values("Name", key=lambda x: ea_count(x.str),  
                      ascending=False)
```

	Name	Sex	Count	Year
414997	Jearlean	F	12	1945
1343738	Chealsea	F	8	2000
298619	Jeanean	F	5	1932
1160221	Seandrea	F	5	1993
976009	Deandrea	M	26	1985
...	...	...	...	...
661810	Santos	F	11	1967
661809	Sandar	F	11	1967
661808	Sam	F	11	1967
661807	Rozetta	F	11	1967
1957045	Zzyzx	M	5	2018

1957046 rows x 4 columns



思考：请用创建一个临时
列的方式实现



提纲

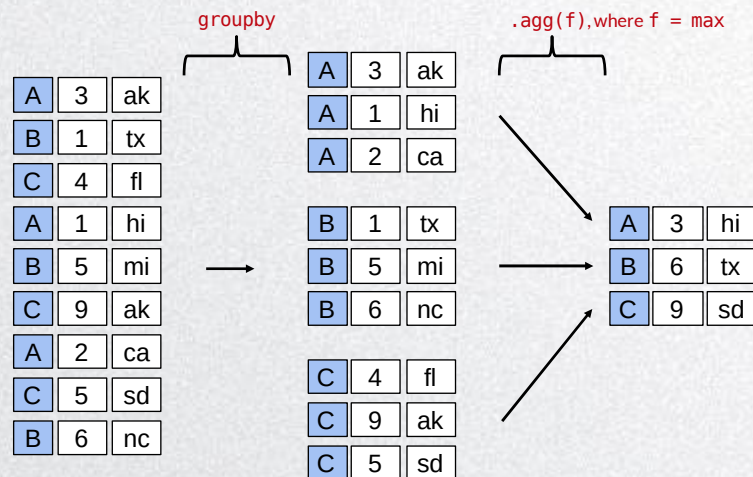
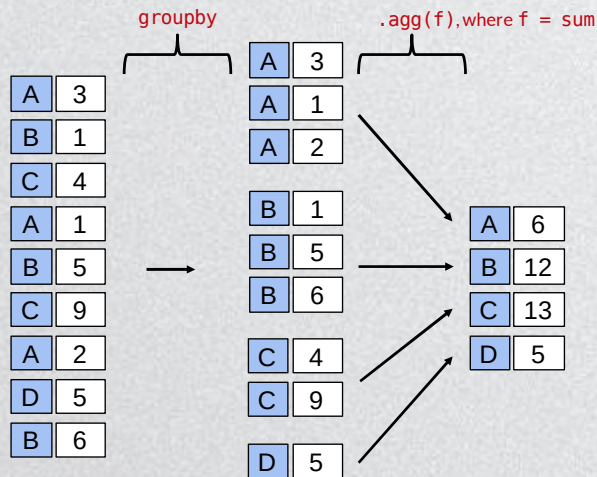


表格数据处理与 Pandas

1. 条件数据筛选
2. 常用数据处理操作
3. 数据分组与数据透视表
4. 案例分析与课堂练习

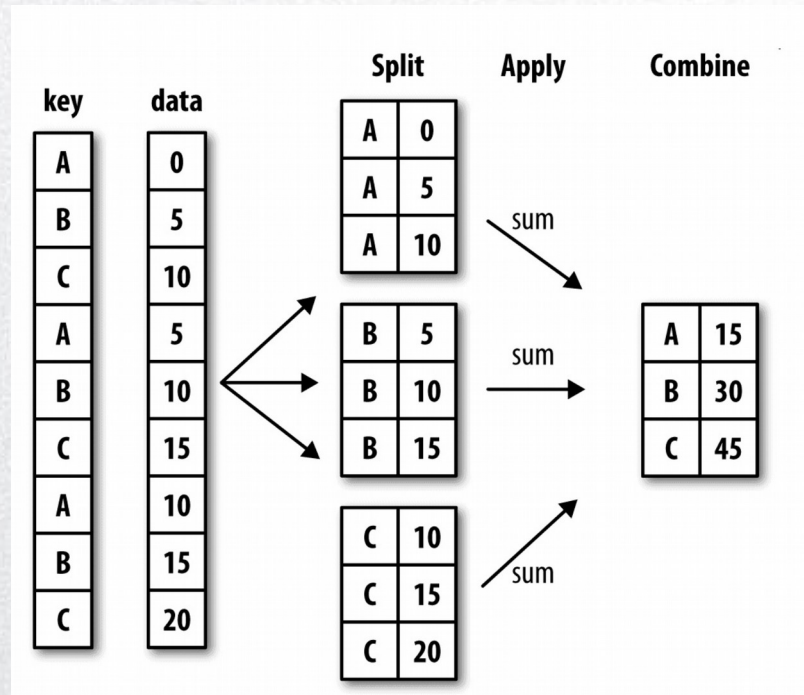
数据聚合操作 -groupby

- 通常我们希望对在某些属性上取值相同的数据点进行聚合分析
 - 每个政党在美国所有选举中的平均投票份额是多少？
 - 中国人民大学每个学期每个系的平均班级人数是多少？
- groupby 是解决这类问题的一个非常强大的工具。



数据聚合操作 -groupby

- 聚合查询 (Aggregate Queries)
 - 使用一个或多个键 (如 DataFrame 列名) 将表格数据进行分组
 - 计算分组的统计信息, 比如数量、平均值或求和
- 聚合函数 (Aggregate Function)
 - count, sum, avg
- 分组机制 (GroupBy)
 - 一个或多个属性
- 选择条件 (Where)
 - 对数据进行筛选





数据聚合操作 -groupby



- 请给出每年新生儿的总数
 - Split 步骤：按照 Year 进行分组
 - Apply 步骤：每组应用 sum 函数
 - Combine 步骤：各组组成一个新的 DataFrame 对象

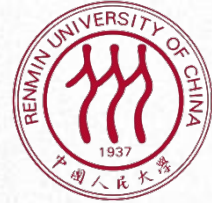
```
babynames.groupby("Year").agg(sum)
```



练习：替换其它的聚合函数

Year	Count
1880	201484
1881	192696
1882	221533
1883	216944
1884	243461
...	...
2014	3698914
2015	3691990
2016	3657797
2017	3561975
2018	3487353

139 rows × 1 column



数据聚合操作 -groupby

The **groupby** method aggregates all rows with the same value for one or more columns into a single row

```
df.groupby(  
    ['nationality', 'degree']  
) .agg(  
    mean_salary = ('salary', 'mean'),  
    min_age = ('age', 'min'),  
    max_age = ('age', 'max')  
) .reset_index()
```

Group by these column

Renamed output columns

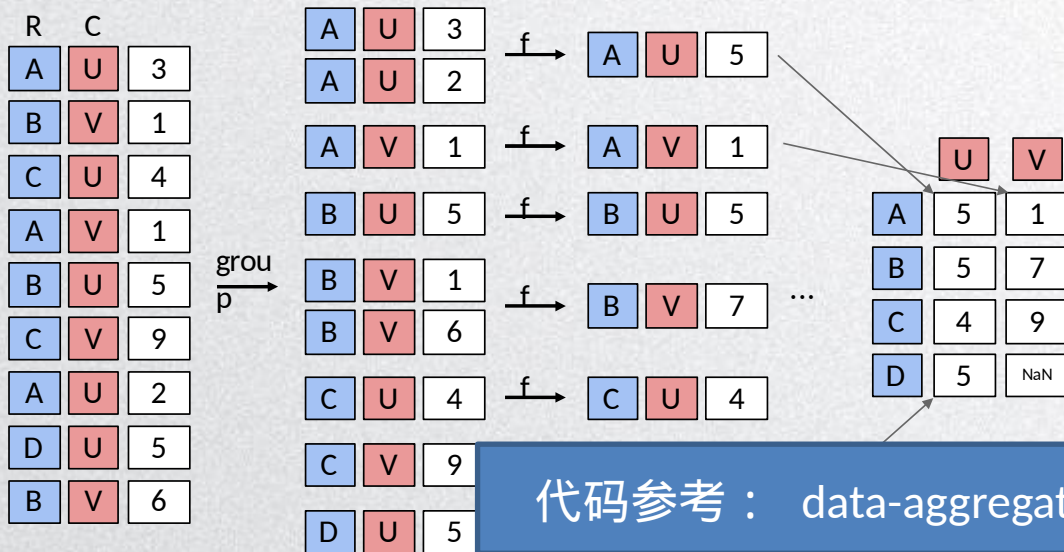
Operation on columns

Selected columns

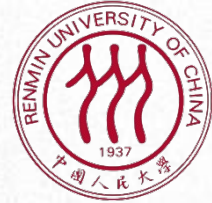
代码参考 : [data-aggregation-group.ipynb](#)

数据透视操作 - pivot_table

- 数据透视表是一种常见于电子表格和数据分析软件中的数据汇总工具。
- 数据透视表通过一个或多个键聚合数据表，将数据排列在一个矩形中，其中一些组键沿着行，一些沿着列。



代码参考：[data-aggregation-group.ipynb](#)



课堂演示：何以知吾名（What's in a Name）

- 问题：体会如何综合运用本节课教授的知识
- 欢迎在线把玩：<https://git.ruc.edu.cn/data-science/notebooks/-/tree/main/us-babynames>

① 数据获取：调查问卷

数据思维课程信息收集

1. 你的中文名

2. 你的英文名（只需给出First Name，如John，无需给出Surname）

*3. 你的性别

☐ 男

☐ 女

英文名	性别
Bruce	男
Nan	女
Joshua	男
Caroline	女
Dora	女
Haidong	男
Cong	男
Amanda	女
Olivia	女
Tian	女
Alice	女
Tars	男
Bess	女

② 数据获取：网络获取

<https://www.ssa.gov/oact/babynames/background.html>

All names are from Social Security card applications for births that occurred in the United States after 1879. Note that many people born before 1937 never applied for a Social Security card, so their names are not included in our data. For others who did apply, our records may not show the place of birth, and again their names are not included in our data.

All data are from a 100% sample of our records on Social Security card applications as of March 2019.

Name	Sex	Count	Year
Mary	F	7065	1880
Anna	F	2604	1880
Emma	F	2003	1880
Elizabeth	F	1939	1880
Minnie	F	1746	1880



提纲



表格数据处理与 Pandas

- 1. 条件数据筛选
- 2. 常用数据处理操作
- 3. 数据分组与数据透视表
- 4. 案例分析与课堂练习



课后练习：Pandas 处理 excel 数据

- 请同学们打开 [Hands-on/exercise-movie-blank.ipynb](#) ，完成代码填空
- 本次作业无需提交，下次上课讲解



课堂练习



- 读取某个用户的所有信息
 - 输入：用户 id
- 找出年龄 >40 岁的所有用户
- 统计少年、青年、中年、老年的人数
 - 少年、青年、中年、老年的年龄范围自定义即可
- 哪个年龄段的人数最多？
- 各个年龄段的男女比例